RELIABLE RAG PLATFORM DEMO HANDBOOK

Document Purpose

This handbook is a sample document created for testing the Reliable RAG Platform. It contains clear facts, dates, module names, constraints, and project decisions so that retrieval, citation generation, and refusal behaviour can be demonstrated during a final year project presentation.

The document describes a fictional deployment of the Reliable RAG Platform for an academic document intelligence use case. It is not an official policy document. It is designed only for local testing and public demo purposes.

Project Overview

The Reliable RAG Platform is a web application that helps users ask questions over uploaded documents. The system combines a FastAPI backend, a React TypeScript frontend, document parsing, chunking, retrieval, Gemini-based answer generation, citation display, and audit logging.

The main goal is to reduce hallucination by forcing the answer generator to use only retrieved evidence from the uploaded corpus. If the platform cannot find enough supporting evidence, the system should reply with the exact refusal sentence: "Not found in the provided documents."

The application was built as an additional final year project by Lakshit Tandon, registration number 220905506, under the internal guidance of Manjula K. Shenoy.

Supported File Types

Version 1 of the platform supports PDF, TXT, and Markdown files. The maximum file size for upload is 10 MB. Files larger than this should be rejected before indexing to prevent slow uploads and poor demo performance.

The system supports text-based PDFs. Scanned PDFs and image-only documents are not supported in version 1 because they require OCR. OCR support is planned as future work.

Ingestion Pipeline

When a user uploads a document, the backend stores the file, calculates a SHA-256 checksum, checks for duplicate content, extracts text, splits the text into chunks, and attaches metadata to every chunk.

Each chunk stores the document name, document id, page number, section title, chunk text, token count, and source path. This metadata is important because the frontend uses it to show citations and source information.

Upload processing runs asynchronously. This means the user can upload a file and the backend can index it in the background while the frontend shows the document status as processing, indexed, or failed.

Retrieval Pipeline

The query pipeline receives a user question and searches the indexed chunks. The prototype uses lexical retrieval and embedding-style scoring concepts to identify relevant text. The original project plan included Qdrant for dense vectors and Whoosh for BM25 keyword retrieval.

The intended production retrieval flow is hybrid retrieval. First, dense retrieval finds semantically similar chunks. Second, BM25 finds keyword-matching chunks. Third, Reciprocal Rank Fusion combines both ranked lists. Fourth, a reranking step selects the strongest evidence chunks. Finally, only the top evidence chunks are passed to the answer generator.

The answer generator must not answer from memory. It should answer only from retrieved chunks. This design makes the system more trustworthy than a plain chatbot because every

answer can be traced back to document evidence.

Answer Generation Rules

The assistant should provide direct answers when evidence is present in the uploaded documents. Every factual answer should include citations. A citation should mention the document name, page number, and section when available.

If retrieved evidence is weak, unrelated, or missing, the assistant should refuse instead of guessing. The required refusal message is: "Not found in the provided documents."

The platform should not invent missing values. For example, if the documents do not mention a project budget, project marks, production users, or cloud database credentials, the assistant should not create those details.

Frontend Modules

The frontend contains a corpus dashboard, a query studio, and an audit trail screen. The corpus dashboard is used for uploading, viewing, and deleting documents. The query studio is used for asking questions and viewing citations. The audit trail is used to show important system events such as uploads, deletions, and queries.

The login screen is currently simplified for demo use. Full registration, role-based authentication, and profile management are planned as future work.

The document preview and metadata panel are also planned as future work. This feature would allow users to open a document, inspect extracted sections, and verify which chunks were used for an answer.

Backend Modules

The backend is implemented as a modular monolith. The main modules are API routes, schemas, domain types, storage service, parsing service, chunking service, retrieval service, model service, security helpers, and configuration.

FastAPI is used because it provides clean REST APIs, automatic validation through Pydantic models, interactive API documentation, and good support for asynchronous request handling.

The backend currently uses in-memory storage for the demo. A persistent PostgreSQL database and durable file storage layer are planned as future work so that uploaded documents and audit logs survive server restarts.

Model Integration

The project includes a provider-aware model layer. Gemini 2.5 Flash Lite is used for answer generation in the deployed prototype because it has a free-tier option and is suitable for lightweight demonstrations.

The system is designed so that the model provider can be changed later without rewriting the entire application. In the future, OpenAI, Gemini, or local models could be plugged into the same query pipeline.

Deployment Notes

The application is deployed on Render at the public URL https://reliable-rag-platform.onrender.com. The deployment uses Docker so that backend and frontend assets can be built in a repeatable way.

For a final year demo, the recommended steps are to upload this handbook, wait for indexing to complete, ask two or three factual questions, inspect citations, ask one negative question, and then show the audit trail.

Recommended Demo Questions

What file types are supported in version 1?

What is the maximum upload size?

Why does the system refuse some questions?

Which frontend modules are currently available?

Who is the internal guide for the project?

Does version 1 support OCR for scanned PDFs?

What database is planned for persistent storage?

Known Limitations

The current prototype does not include OCR, full registration, enterprise SSO, multi-tenant user isolation, or a production-grade database. These limitations are intentional because the project is scoped as a solo-student final year additional project.

The current evaluation benchmark module was removed from the user-facing demo to avoid misleading fixed benchmark scores. Instead, the project focuses on live document upload, retrieval, citations, grounded answering, and auditability.

Future Enhancements

The planned future work includes OCR for scanned PDFs, PostgreSQL persistence, document preview, metadata inspection, admin analytics, query history, improved role-based access control, larger corpus testing, and more advanced retrieval evaluation.

The most important future improvement is persistent storage because it would make the application more reliable after redeployment or server restart.